

## Multiple Choice Questions

1. What keyword is required to define a method as asynchronous in C#?
  - a) `await`
  - b) `async`
  - c) `Task`
  - d) `run`
2. Which of the following is a correct return type for an asynchronous method?
  - a) `string`
  - b) `Thread`
  - c) `Task`
  - d) `Runnable`
3. What does the `await` keyword do?
  - a) Immediately returns from the method
  - b) Blocks the calling thread
  - c) Suspends the method until the awaited task completes
  - d) Runs the method synchronously
4. Which class is used for non-blocking HTTP requests?
  - a) `WebClient`
  - b) `HttpRequest`
  - c) `HttpClient`
  - d) `SocketClient`
5. What method would you use to asynchronously read a line from a file?
  - a) `ReadLine()`
  - b) `ReadAsync()`
  - c) `ReadLineAsync()`
  - d) `ReadString()`
6. What will `Task.Run()` do?
  - a) Start a new UI thread
  - b) Run code on the thread pool
  - c) Delay the task
  - d) Pause the main thread
7. Which of these will block the current thread?
  - a) `await`
  - b) `Task.Run()`
  - c) `task.Result`
  - d) `async`
8. Which of the following is true about `Task.Delay()`?

- a) It delays the thread execution
  - b) It blocks the thread
  - c) It introduces a non-blocking wait
  - d) It cancels the task
9. What is the primary benefit of async methods in a GUI application?
- a) Faster computation
  - b) Improved responsiveness
  - c) Reduced memory use
  - d) Shorter code
10. What type of operations benefit most from asynchronous programming?
- a) Integer multiplication
  - b) Sorting arrays
  - c) I/O-bound operations
  - d) Floating-point division
11. What happens if you call `.Result` on a task in a UI thread?
- a) It completes the task faster
  - b) It improves performance
  - c) It may cause a deadlock
  - d) It makes the app crash immediately
12. What is the role of `ThreadPool.QueueUserWorkItem()`?
- a) Starts a new UI thread
  - b) Schedules work on a thread pool
  - c) Blocks the calling thread
  - d) Reads data from a file
13. Which method is used to wait for multiple tasks to complete?
- a) `await All()`
  - b) `WaitAll()`
  - c) `Task.WhenAll()`
  - d) `JoinTasks()`
14. Which keyword is needed to use `await` in a method?
- a) `using`
  - b) `static`
  - c) `async`
  - d) `new`
15. What type does `HttpClient.GetStringAsync()` return?
- a) `string`
  - b) `Task`

- c) `Task<string>`
- d) `void`

16. In the example with Stopwatch, what is measured?

- a) The memory used by tasks
- b) The network speed
- c) The execution time of multiple operations
- d) The size of the HTTP content

17. What would be the output type of `CalculateSumOfRandsAsync()`?

- a) `long`
- b) `int`
- c) `Task<long>`
- d) `void`

18. What happens when a method is marked as `async` but does not contain `await`?

- a) Compilation error
- b) It runs synchronously
- c) It blocks the thread
- d) It runs faster

19. Which of the following is **not** a good practice in async programming?

- a) Using `await`
- b) Avoiding `.Result`
- c) Using `async void` everywhere
- d) Returning `Task` from async methods

20. Which class is used for reading text files line by line asynchronously?

- a) `StreamWriter`
- b) `FileReader`
- c) `StreamReader`
- d) `TextFileReader`

21. Which method is the asynchronous version of `SumAll(int[] values)`?

- a) `SumAllTask()`
- b) `SumAllAsync()`
- c) `AsyncSumAll()`
- d) `SumAllAwaited()`

22. What is a common symptom of blocking the UI thread?

- a) Faster execution
- b) Thread reuse
- c) Application freezing
- d) Better memory usage

23. Which statement is true about `async void` methods?
- a) They are ideal for async logic
  - b) They support exception handling
  - c) They are suitable only for event handlers
  - d) They return values
24. What is the result of calling an asynchronous method and not awaiting it?
- a) The method never starts
  - b) It causes a compiler error
  - c) The task runs without waiting for completion
  - d) It blocks the caller
25. Which is a correct way to define an asynchronous method returning an integer?
- a) `async int Method()`
  - b) `int async Method()`
  - c) `async Task<int> Method()`
  - d) `async Method<int>()`
26. What does `await GetStringTask` do?
- a) Cancels the task
  - b) Retrieves the result asynchronously
  - c) Forces parallel execution
  - d) Blocks until it's scheduled
27. What happens if you call multiple `async` methods with `await` one after another?
- a) They execute in parallel
  - b) They execute sequentially
  - c) They execute in reverse order
  - d) None of the above
28. How can you start `async` methods in parallel?
- a) `await` each call immediately
  - b) Use `Task.WhenAll()`
  - c) Use `Thread.Sleep()`
  - d) Wrap all calls in `async void`
29. Which of these is an example of converting a synchronous method to `async`?
- a) `int ReadData() → int ReadDataAsync()`
  - b) `void Read() → async Read()`
  - c) `byte[] ReadData() → Task<byte[]> ReadDataAsync()`
  - d) `int Read() → Awaitable<int> Read()`
30. What happens when `GetUrlContentLengthAsync().Result` is called?
- a) The app crashes
  - b) The result is awaited correctly
  - c) The method is run synchronously and may block
  - d) It runs the task in a new thread

## Answer Key

1. b) `async`
2. c) `Task`
3. c) Suspends the method until the awaited task completes
4. c) `HttpClient`
5. c) `ReadLineAsync()`
6. b) Run code on the thread pool
7. c) `task.Result`
8. c) It introduces a non-blocking wait
9. b) Improved responsiveness
10. c) I/O-bound operations
11. c) It may cause a deadlock
12. b) Schedules work on a thread pool
13. c) `Task.WhenAll()`
14. c) `async`
15. c) `Task<string>`
16. c) The execution time of multiple operations
17. c) `Task<long>`
18. b) It runs synchronously
19. c) Using `async void` everywhere
20. c) `StreamReader`
21. b) `SumAllAsync()`
22. c) Application freezing
23. c) They are suitable only for event handlers
24. c) The task runs without waiting for completion
25. c) `async Task<int> Method()`
26. b) Retrieves the result asynchronously
27. b) They execute sequentially
28. b) Use `Task.WhenAll()`
29. c) `byte[] ReadData() → Task<byte[]> ReadDataAsync()`
30. c) The method is run synchronously and may block

## ✓ Multiple-Answer Questions (Select All That Apply)

1. Which of the following are characteristics of synchronous methods?
  - ☐ They block the caller
  - ☐ They return a Task
  - ☐ They execute sequentially
  - ☐ They allow continuation without waiting
2. What makes a method asynchronous?
  - ☐ It includes the async keyword
  - ☐ It performs operations in parallel automatically
  - ☐ It uses await
  - ☐ It returns immediately without blocking
3. Which return types are valid for an async method?
  - ☐ void
  - ☐ Task
  - ☐ int
  - ☐ Task<T>
4. Which methods can be used to run work on a background thread?
  - ☐ Thread.Sleep
  - ☐ ThreadPool.QueueUserWorkItem
  - ☐ Task.Run
  - ☐ await
5. Which methods are asynchronous versions of I/O operations?
  - ☐ ReadLineAsync()
  - ☐ WriteLineAsync()
  - ☐ GetStringAsync()
  - ☐ Console.ReadLine()
6. What does await do in an async method?
  - ☐ Blocks the current thread
  - ☐ Allows the method to resume when the awaited task is complete
  - ☐ Enables non-blocking behavior
  - ☐ Automatically runs the task in parallel
7. Which of these are valid ways to start multiple asynchronous tasks?
  - ☐ Start them with await inside a loop
  - ☐ Store them in Task variables and await later
  - ☐ Use Task.WhenAll()
  - ☐ Use Thread.Sleep
8. Which statements about HttpClient.GetStringAsync() are true?
  - ☐ It is a built-in async method
  - ☐ It downloads string content from a URL
  - ☐ It blocks until the request is complete
  - ☐ It returns a Task<string>
9. What happens when you use .Result on a Task?
  - ☐ The thread waits for the task to finish
  - ☐ It may cause a deadlock in UI applications
  - ☐ It always runs the task on a new thread
  - ☐ It turns an async call into a blocking call

10. What are the advantages of using `async/await`?
  - ☐ Better responsiveness
  - ☐ Automatic parallel execution of all code
  - ☐ Efficient use of threads
  - ☐ Simplified asynchronous code structure
11. Which statements about `CalculateSumOfRandsAsync` are true?
  - ☐ It uses `Task.Run`
  - ☐ It returns a `Task<long>`
  - ☐ It blocks until the result is available
  - ☐ It performs CPU-bound work
12. Which of these methods are synchronous?
  - ☐ `CalculateSumOfRands`
  - ☐ `Console.WriteLine`
  - ☐ `GetStringAsync`
  - ☐ `ReadLine`
13. What should you avoid doing inside a GUI thread?
  - ☐ Blocking with `.Result`
  - ☐ Using `await`
  - ☐ Running time-consuming tasks
  - ☐ Updating UI from background threads
14. Which of the following statements are true about `Task.Delay(2000)`?
  - ☐ It pauses the thread for 2 seconds
  - ☐ It simulates a long-running async operation
  - ☐ It returns immediately
  - ☐ It's useful in demonstration or testing scenarios
15. What can be used to measure how long an operation takes?
  - ☐ Stopwatch
  - ☐ `DateTime.Now`
  - ☐ `TimeSpan.FromMilliseconds()`
  - ☐ `Task.Delay`
16. What is true about `Task.WhenAll()`?
  - ☐ It runs all tasks in parallel
  - ☐ It returns a single `Task` representing all completions
  - ☐ It waits for tasks one by one
  - ☐ It can be awaited
17. Which are valid asynchronous file reading patterns?
  - ☐ `await reader.ReadLineAsync()`
  - ☐ `reader.ReadLine()`
  - ☐ `File.ReadAllText()`
  - ☐ `File.ReadAllTextAsync()`
18. Which methods are valid for asynchronous writing?
  - ☐ `StreamWriter.WriteLineAsync()`
  - ☐ `Console.WriteLine()`
  - ☐ `File.WriteAllTextAsync()`
  - ☐ `WriteAsync()`
19. What causes deadlocks in async code?
  - ☐ Using `.Result` inside the UI thread
  - ☐ Blocking the synchronization context

- ☐ Multiple await statements
  - ☐ Using async void
20. What are good use cases for async programming?
- ☐ CPU-bound processing
  - ☐ Downloading web content
  - ☐ Reading large files
  - ☐ Performing I/O without blocking
21. Which are examples of I/O-bound operations?
- ☐ Writing to a file
  - ☐ Querying a database
  - ☐ Parsing a large string
  - ☐ Sending an HTTP request
22. What are features of the Task class?
- ☐ Represents asynchronous operation
  - ☐ Has a Result property
  - ☐ Must always return void
  - ☐ Can be awaited
23. When calling GetUrlContentLengthAsync, what happens?
- ☐ It returns a Task<int>
  - ☐ It downloads a webpage
  - ☐ It blocks the thread
  - ☐ It uses await internally
24. What does await Task.WhenAll(t1, t2, t3) do?
- ☐ Runs tasks sequentially
  - ☐ Waits for all three tasks to complete
  - ☐ Returns when the first task completes
  - ☐ Can be used for parallelism
25. Which of these can be considered best practices in async programming?
- ☐ Use ConfigureAwait(false) in libraries
  - ☐ Prefer async Task over async void
  - ☐ Avoid .Result unless necessary
  - ☐ Always use Task.Run even for I/O
26. What are the downsides of using async void?
- ☐ Difficult to test
  - ☐ Doesn't support await
  - ☐ Can't return exceptions properly
  - ☐ Only appropriate for event handlers
27. Which of these classes support asynchronous operations?
- ☐ HttpClient
  - ☐ StreamReader
  - ☐ Console
  - ☐ StreamWriter
28. What statements are true about thread pools?
- ☐ Threads are reused
  - ☐ Tasks can be scheduled efficiently
  - ☐ Every Task.Run creates a new OS thread
  - ☐ Avoids the overhead of creating new threads

29. In the example with Stopwatch, what is being measured?
- ☐ Total time of all async operations
  - ☐ File reading speed
  - ☐ Network download time
  - ☐ Time spent on UI rendering
30. Which options make code asynchronous?
- ☐ Using Task.Run()
  - ☐ Adding async and await
  - ☐ Using blocking I/O
  - ☐ Calling asynchronous API methods

## Answer Key (Selected Correct Answers)

1. ✓ They block the caller, ✓ They execute sequentially
2. ✓ It includes the async keyword, ✓ It uses await, ✓ It returns immediately without blocking
3. ✓ void, ✓ Task, ✓ Task<T>
4. ✓ ThreadPool.QueueUserWorkItem, ✓ Task.Run
5. ✓ ReadLineAsync(), ✓ WriteLineAsync(), ✓ GetStringAsync()
6. ✓ Allows the method to resume..., ✓ Enables non-blocking behavior
7. ✓ Store them in Task variables..., ✓ Use Task.WhenAll()
8. ✓ It is a built-in async method, ✓ It downloads string content..., ✓ It returns a Task<string>
9. ✓ The thread waits..., ✓ It may cause a deadlock..., ✓ It turns an async call into...
10. ✓ Better responsiveness, ✓ Efficient use of threads, ✓ Simplified async code structure
11. ✓ It uses Task.Run, ✓ It returns a Task<long>, ✓ It performs CPU-bound work
12. ✓ CalculateSumOfRands, ✓ Console.WriteLine, ✓ ReadLine
13. ✓ Blocking with .Result, ✓ Running time-consuming tasks, ✓ Updating UI from background threads
14. ✓ It simulates a long-running..., ✓ It's useful in demonstration...
15. ✓ Stopwatch, ✓ DateTime.Now
16. ✓ It runs all tasks in parallel, ✓ It returns a single Task..., ✓ It can be awaited
17. ✓ await reader.ReadLineAsync(), ✓ File.ReadAllTextAsync()
18. ✓ StreamWriter.WriteLineAsync(), ✓ File.WriteAllTextAsync(), ✓ WriteAsync()
19. ✓ Using .Result inside the UI thread, ✓ Blocking the synchronization..., ✓ Using async void
20. ✓ Downloading web content, ✓ Reading large files, ✓ Performing I/O without blocking
21. ✓ Writing to a file, ✓ Querying a database, ✓ Sending an HTTP request
22. ✓ Represents asynchronous operation, ✓ Has a Result property, ✓ Can be awaited
23. ✓ It returns a Task<int>, ✓ It downloads a webpage, ✓ It uses await internally
24. ✓ Waits for all three tasks to complete, ✓ Can be used for parallelism
25. ✓ Use ConfigureAwait(false), ✓ Prefer async Task over async void, ✓ Avoid .Result unless necessary
26. ✓ Difficult to test, ✓ Can't return exceptions properly, ✓ Only appropriate for event handlers
27. ✓ HttpClient, ✓ StreamReader, ✓ StreamWriter
28. ✓ Threads are reused, ✓ Tasks can be scheduled efficiently, ✓ Avoids the overhead...
29. ✓ Total time of all async operations, ✓ Network download time
30. ✓ Using Task.Run(), ✓ Adding async and await, ✓ Calling asynchronous API methods

## True or False Questions

1. In a synchronous operation, the caller can continue executing while the operation is still running.
2. The method `BeginSort(data)` is asynchronous if it returns immediately without waiting for sorting to finish.
3. `Task.Run` executes the given delegate on the current UI thread.
4. Using `.Result` on a `Task` in a UI thread can cause the application to freeze.
5. `CalculateSumOfRandsAsync` returns a value of type `void`.
6. The `async` keyword can be used with any method, even if it does not contain any `await`.
7. Using `await` inside an `async` method blocks the thread until the result is ready.
8. `Task.Delay(2000)` is used to simulate a time-consuming operation asynchronously.
9. The `GetUrlContentLength` method uses asynchronous code correctly.
10. You can use `StreamReader.ReadLine()` inside an `async` method without any issues.
11. `HttpClient.GetStringAsync()` is a built-in asynchronous method in .NET.
12. Asynchronous programming allows for better responsiveness in GUI applications.
13. `ThreadPool.QueueUserWorkItem` is used to offload tasks to the thread pool.
14. The `Stopwatch` class can be used to measure the execution time of `async` tasks.
15. Awaiting tasks one after another allows all of them to run in parallel.
16. `Task.WhenAll` is used to wait for multiple tasks to complete concurrently.
17. `GetLongestLineLengthAsync` reads the file synchronously.
18. `Task<int>` represents a task that returns an integer value in the future.
19. Blocking a task using `.Wait()` or `.Result` is not recommended in `async` code.
20. `CalculateSumOfRands` and `CalculateSumOfRandsAsync` both use the `Random` class.
21. Tasks created with `Task.Run` are scheduled on the thread pool.
22. Asynchronous operations are best suited for CPU-intensive calculations.
23. You can use `await` outside of an `async` method as long as it returns a `Task`.
24. `GetUrlContentLengthAsync` properly uses `await` for both HTTP calls and delays.
25. `ReadLineAsync()` is a non-blocking method in `StreamReader`.
26. Using multiple `await` statements in sequence causes the operations to run in parallel.
27. Asynchronous code in .NET is often used for I/O-bound operations.
28. The return type of an `async` method must be `Task`, `Task<T>`, or `void`.
29. Declaring `Task<int> t1 = GetUrlContentLengthAsync(url1);` immediately starts the task.
30. Calling `GetUrlContentLengthAsync().Result` effectively converts it to a synchronous operation.

# Answer Key

1. False
2. True
3. False
4. True
5. False
6. True
7. False
8. True
9. False
10. True
11. True
12. True
13. True
14. True
15. False
16. True
17. False
18. True
19. True
20. True
21. True
22. False
23. False
24. True
25. True
26. False
27. True
28. True
29. True
30. True

# Complete the Sentence Questions

1. A synchronous operation is one in which the caller \_\_\_\_\_ until the operation completes.
2. An asynchronous operation begins the task and \_\_\_\_\_ immediately.
3. In C#, the Task type is used to represent an operation that will \_\_\_\_\_ in the future.
4. The method CalculateSumOfRandsAsync returns a task of type \_\_\_\_\_.
5. ThreadPool.QueueUserWorkItem is used to \_\_\_\_\_ a method on a background thread.
6. Using .Result on a Task can cause the application to \_\_\_\_\_.
7. An async method typically returns a Task, Task<T>, or \_\_\_\_\_.
8. To wait for the result of an asynchronous method, you should use the \_\_\_\_\_ keyword.
9. The method BeginSort (data) is asynchronous if it only starts sorting and then \_\_\_\_\_.
10. ReadLineAsync () is an asynchronous version of the method \_\_\_\_\_.
11. The await keyword can only be used inside methods marked with the \_\_\_\_\_ keyword.
12. Asynchronous file reading in C# can be performed using the class \_\_\_\_\_.
13. To simulate a delay in an asynchronous method, you can use Task.\_\_\_\_\_.
14. Using HttpClient.GetStringAsync () inside an async method will return a Task<\_\_\_\_\_.
15. You should avoid blocking asynchronous operations with \_\_\_\_\_ or .Wait ().
16. In an asynchronous GUI app, using blocking code can cause the UI to \_\_\_\_\_.
17. Asynchronous programming helps improve responsiveness by allowing other work to continue while waiting for \_\_\_\_\_.
18. The CalculateSumOfRands method is CPU-bound, so using Task.Run helps offload work to a \_\_\_\_\_ thread.
19. The method GetLongestLineLengthAsync asynchronously reads lines using \_\_\_\_\_.
20. To start multiple asynchronous tasks concurrently, you should not await them immediately but instead store them in \_\_\_\_\_.
21. The expression await Task.WhenAll (t1, t2, t3) is used to wait for \_\_\_\_\_.
22. Asynchronous programming is especially useful for \_\_\_\_\_-bound operations.
23. As shown in the GetContentLengthButton\_Click method, asynchronous calls can be made for multiple URLs using \_\_\_\_\_.
24. Avoid using Result inside a UI thread, as it can lead to \_\_\_\_\_.
25. GetUrlContentLengthAsync uses Task.Delay (2000) to simulate a(n) \_\_\_\_\_ operation.
26. A major advantage of async methods is improved app performance and better use of \_\_\_\_\_.
27. The Stopwatch class is used to measure the \_\_\_\_\_ of operations.

28. The `HttpClient` class is typically used for making \_\_\_\_\_ requests.
29. The correct way to write an asynchronous version of `int SumAll(int[] values)` is `Task<_____> SumAllAsync(...)`.
30. If you await each task one after another instead of concurrently, the total operation time will be \_\_\_\_\_.

## Answer Key

1. waits
2. returns
3. complete
4. Task
5. queue
6. block
7. void
8. await
9. returns
10. `ReadLine()`
11. `async`
12. `StreamReader`
13. `Delay`
14. `string`
15. `.Result`
16. `freeze`
17. I/O completion
18. `background`
19. `ReadLineAsync()`
20. variables
21. all tasks to finish
22. I/O
23. multiple Task objects
24. deadlocks
25. time-consuming
26. system resources
27. duration

28. HTTP

29. int

30. longer